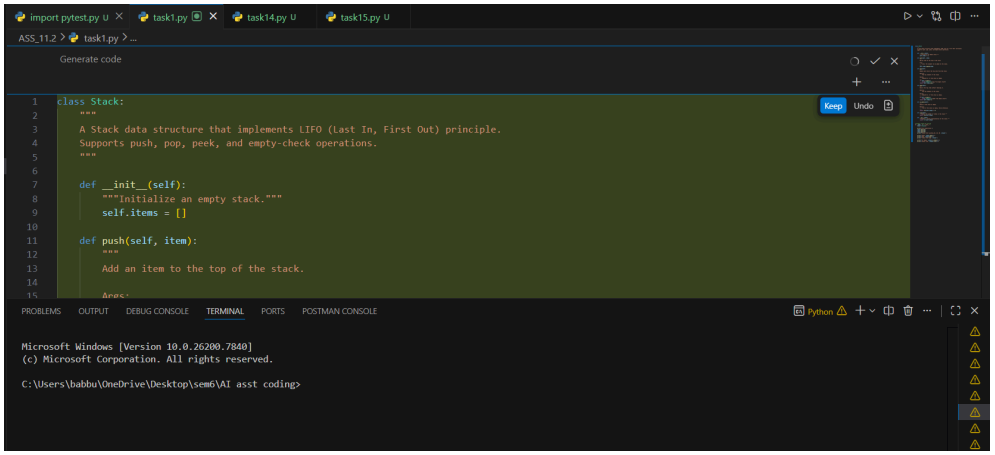
ASSIGNMENT 11.2
AI ASSISTED CODING

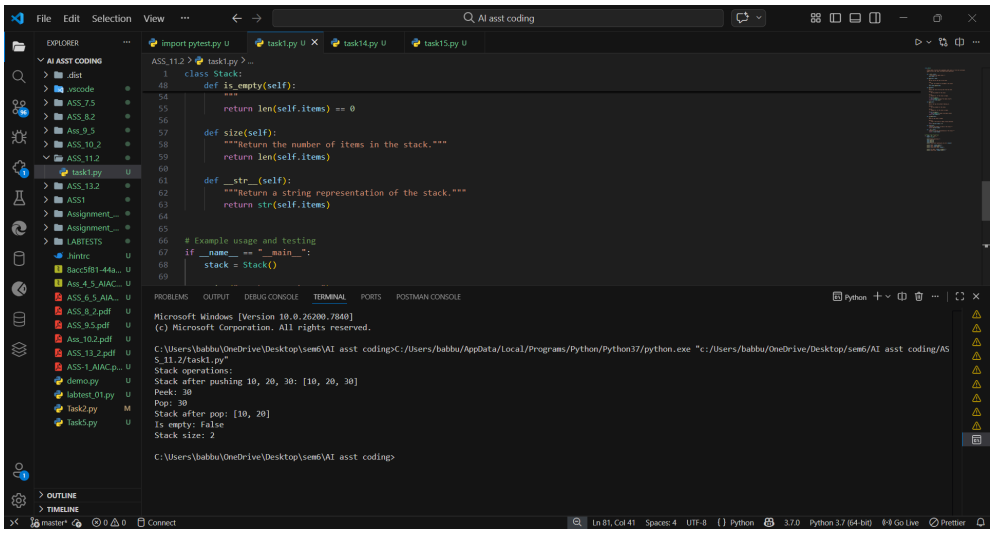
E.SAI ANURATH | BATCH-29 | 2303A51560 |
DATE:10-03-2026

> **Task Description -1 – (Stack Using AI Guidance)**
> **• Task: With the help of AI, design and implement a Stack data structure supporting basic stack operations.**
> **Expected Output:**
> **• A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.**

```
class Stack:
    """
    A Stack data structure that implements LIFO (Last In, First Out) principle.
    Supports push, pop, peek, and empty-check operations.
    """

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """
        Add an item to the top of the stack.

        Args:
            item: The element to be added to the stack.
        """
        self.items.append(item)

    def pop(self):
        """
        Remove and return the top item from the stack.

        Returns:
            The top element of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self.items.pop()

    def peek(self):
        """
        Return the top item without removing it.

        Returns:
            The top element of the stack.
```

```
        Raises:
            IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self.items[-1]

    def is_empty(self):
        """
        Check if the stack is empty.

        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self.items)

    def __str__(self):
        """Return a string representation of the stack."""
        return str(self.items)


# Example usage and testing
if __name__ == "__main__":
    stack = Stack()

    print("Stack operations:")
    stack.push(10)
    stack.push(20)
    stack.push(30)
    print(f"Stack after pushing 10, 20, 30: {stack}")

    print(f"Peek: {stack.peek()}")
    print(f"Pop: {stack.pop()}")
    print(f"Stack after pop: {stack}")

    print(f"Is empty: {stack.is_empty()}")
    print(f"Stack size: {stack.size()}")
```





EXPLAINATION

A **Stack** is a linear data structure that follows the **LIFO principle (Last In, First Out)**. This means the element inserted last will be the first element removed.
A common real-life example of a stack is a **stack of plates**. When plates are placed one above another, the last plate placed on top is the first one removed.
In this task, a Stack class is implemented in Python with the help of AI guidance. The stack supports basic operations such as:

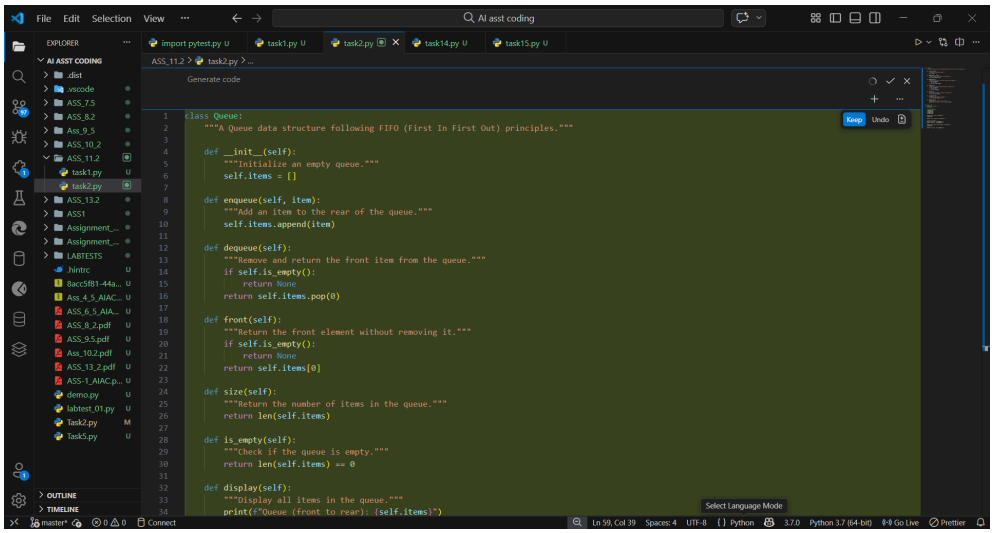
- **Push** – Adds an element to the top of the stack.
- **Pop** – Removes the top element from the stack.
- **Peek** – Displays the top element without removing it.
- **isEmpty** – Checks whether the stack is empty.

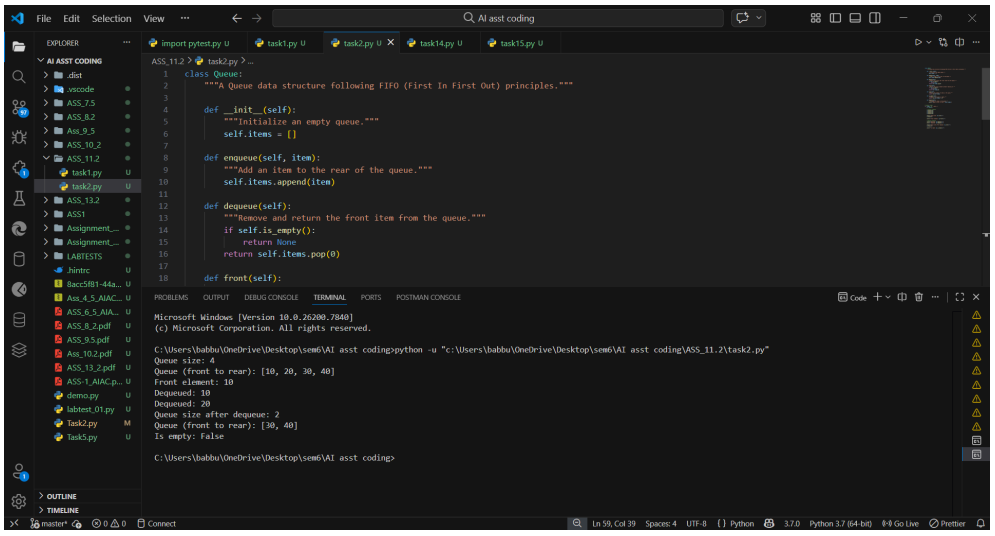
The stack is implemented using a **Python list**, where elements are added and removed from the end of the list to maintain the LIFO behavior.

> **Task Description -2 – (Queue Design)**
> **• Task:Use AI assistance to create a Queue data structure following FIFO principles**
> **Expected Output:**
> **• A complete Queue implementation including enqueue, dequeue, front element access, and size calculation**

```python
class Queue:
    """A Queue data structure following FIFO (First In First Out) principles."""

    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an item to the rear of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the front item from the queue."""
        if self.is_empty():
            return None
        return self.items.pop(0)

    def front(self):
        """Return the front element without removing it."""
        if self.is_empty():
            return None
        return self.items[0]

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.items) == 0

    def display(self):
        """Display all items in the queue."""
        print(f"Queue (front to rear): {self.items}")


# Example usage
if __name__ == "__main__":
    q = Queue()

    # Enqueue elements
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)
    q.enqueue(40)

    print(f"Queue size: {q.size()}")
    q.display()

    print(f"Front element: {q.front()}")

    # Dequeue elements
    print(f"Dequeued: {q.dequeue()}")
    print(f"Dequeued: {q.dequeue()}")
```

```
print(f"Queue size after dequeue: {q.size()}")
q.display()

print(f"Is empty: {q.is_empty()}")
```





EXPLAINATION

💡 A **Queue** is a linear data structure that follows the **FIFO principle (First In, First Out)**. This means the element inserted first will be the first element removed.
A real-life example of a queue is a **line of people waiting at a ticket counter**. The person who arrives first gets the ticket first.
In this task, a Queue data structure is implemented with the help of AI. The queue performs operations such as:

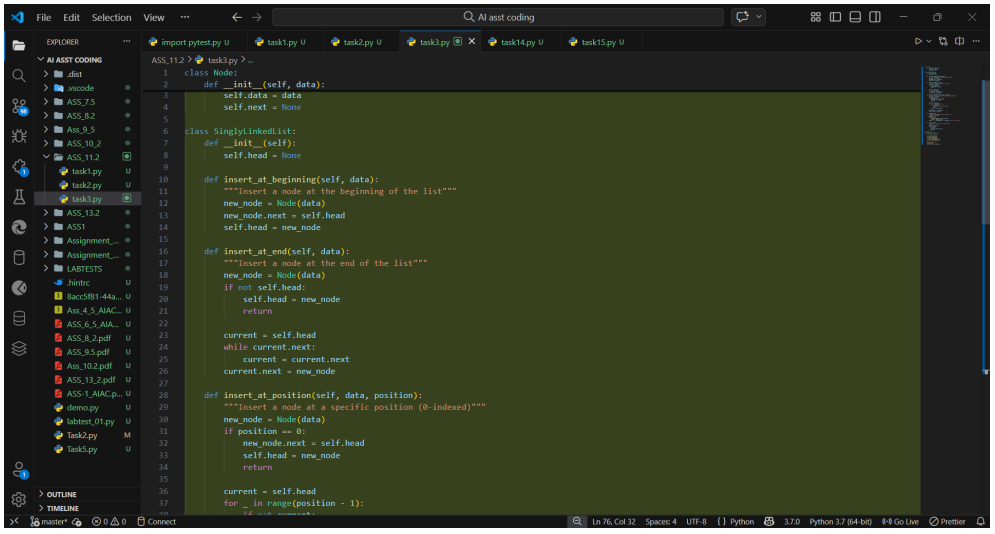
- **Enqueue** – Adds an element to the rear of the queue.
- **Dequeue** – Removes an element from the front of the queue.
- **Front** – Displays the element at the front of the queue.
- **Size** – Calculates the number of elements present in the queue.

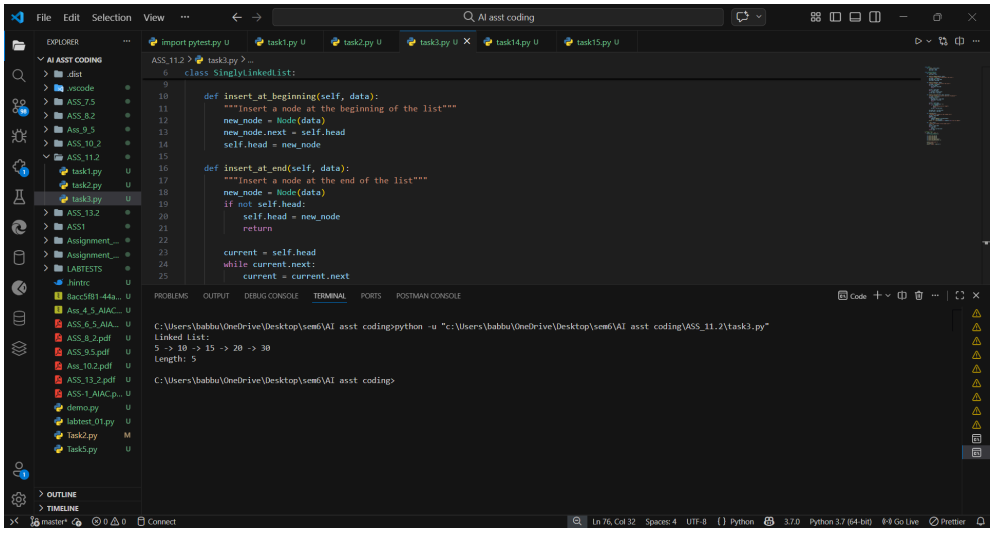
The queue ensures that elements are processed in the same order they are inserted.

> **Task Description -3 –(Singly Linked List Construction)**
> **• Task: Utilize AI to build a singly linked list supporting insertion and traversal.**
> **Expected Output:**
> **• Correctly functioning linked list with node creation, insertion logic, and display functionality.**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        """Insert a node at the beginning of the list"""
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, data):
        """Insert a node at the end of the list"""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def insert_at_position(self, data, position):
        """Insert a node at a specific position (0-indexed)"""
        new_node = Node(data)
        if position == 0:
            new_node.next = self.head
```

```
            self.head = new_node
            return

        current = self.head
        for _ in range(position - 1):
            if not current:
                print("Position out of range")
                return
            current = current.next

        new_node.next = current.next
        current.next = new_node

    def traverse(self):
        """Display all elements in the linked list"""
        current = self.head
        elements = []
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements) if elements else "List is empty")

    def __len__(self):
        """Return the length of the linked list"""
        count = 0
        current = self.head
        while current:
            count += 1
            current = current.next
        return count

# Example usage
if __name__ == "__main__":
    ll = SinglyLinkedList()

    ll.insert_at_end(10)
    ll.insert_at_end(20)
    ll.insert_at_end(30)
    ll.insert_at_beginning(5)
    ll.insert_at_position(15, 2)

    print("Linked List:")
    ll.traverse()
    print(f"Length: {len(ll)}")
```





EXPLAINATION

A **Singly Linked List** is a dynamic data structure consisting of nodes. Each node contains two parts:

- **Data** – Stores the value.
- **Next pointer** – Points to the next node in the list.

Unlike arrays, linked lists do not store elements in contiguous memory locations. Instead, each node points to the next node, forming a chain-like structure.
In this task, a singly linked list is implemented with the help of AI guidance. The program performs the following operations:

- **Node creation** – Creating a node with data and a reference to the next node.
- **Insertion** – Adding new nodes to the linked list.
- **Traversal** – Displaying all elements by moving from one node to another.

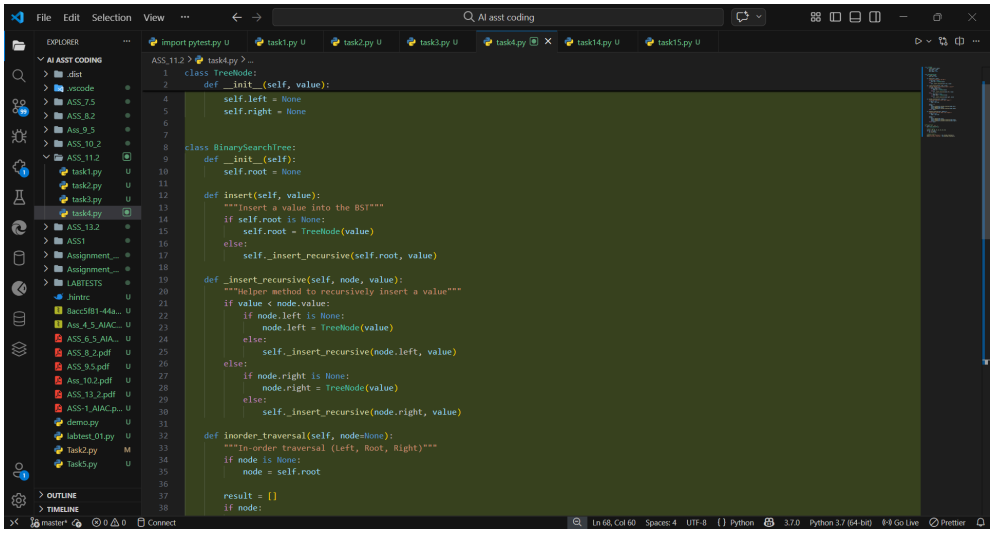
Linked lists are useful when the size of the data changes frequently because they allow **dynamic memory allocation**.

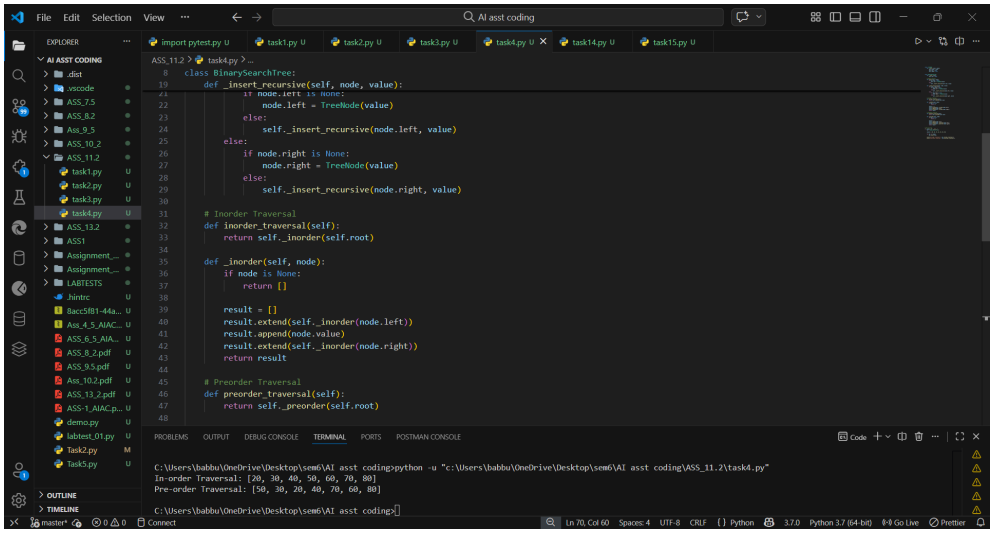
> **Task Description -4 – (Binary Search Tree Operations)**
> **• Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.**

> **Expected Output:**
> **• BST program with correct node insertion and in-order traversal output.**

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None


class BinarySearchTree:
    def __init__(self):
        self.root = None

    # Insert Function
    def insert(self, value):
        if self.root is None:
            self.root = TreeNode(value)
        else:
            self._insert_recursive(self.root, value)

    def _insert_recursive(self, node, value):
        if value < node.value:
            if node.left is None:
                node.left = TreeNode(value)
            else:
                self._insert_recursive(node.left, value)
        else:
            if node.right is None:
                node.right = TreeNode(value)
            else:
                self._insert_recursive(node.right, value)

    # Inorder Traversal
    def inorder_traversal(self):
        return self._inorder(self.root)

    def _inorder(self, node):
        if node is None:
            return []

        result = []
        result.extend(self._inorder(node.left))
        result.append(node.value)
        result.extend(self._inorder(node.right))
        return result

    # Preorder Traversal
    def preorder_traversal(self):
        return self._preorder(self.root)

    def _preorder(self, node):
        if node is None:
            return []
```

```
        result = []
        result.append(node.value)
        result.extend(self._preorder(node.left))
        result.extend(self._preorder(node.right))
        return result


# Main Program
if __name__ == "__main__":
    bst = BinarySearchTree()

    values = [50, 30, 70, 20, 40, 60, 80]

    for val in values:
        bst.insert(val)

    print("In-order Traversal:", bst.inorder_traversal())
    print("Pre-order Traversal:", bst.preorder_traversal())
```





EXPLAINATION

💡 A **Binary Search Tree (BST)** is a hierarchical data structure in which each node has at most two children:

- **Left child**
- **Right child**

The main property of a Binary Search Tree is:

- All elements in the **left subtree are smaller** than the root node.
- All elements in the **right subtree are greater** than the root node.

This structure allows efficient searching, insertion, and traversal operations.
In this task, a BST is implemented using AI assistance. The program includes:

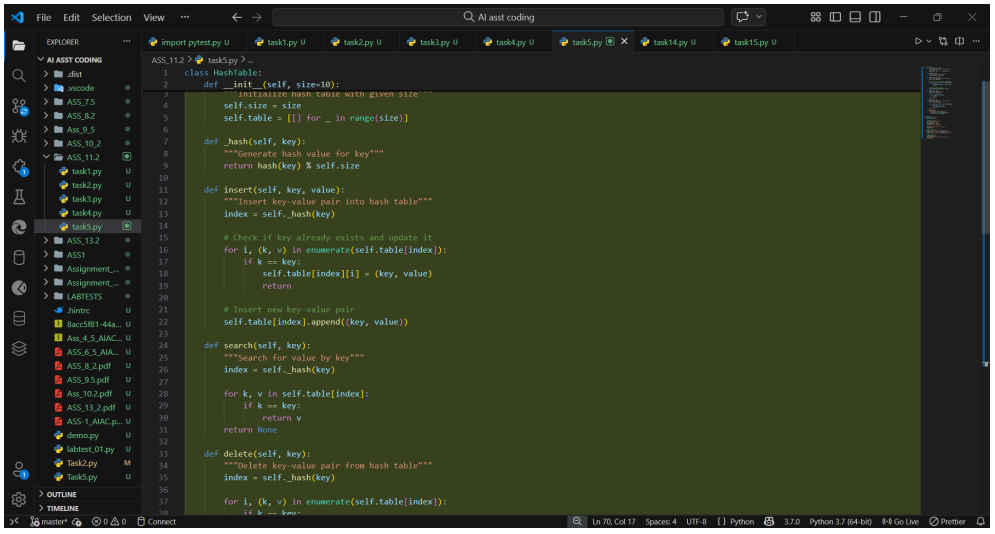
- **Node insertion** – Adding elements according to BST rules.
- **In-order traversal** – Visiting nodes in the order **Left** → **Root** → **Right**.

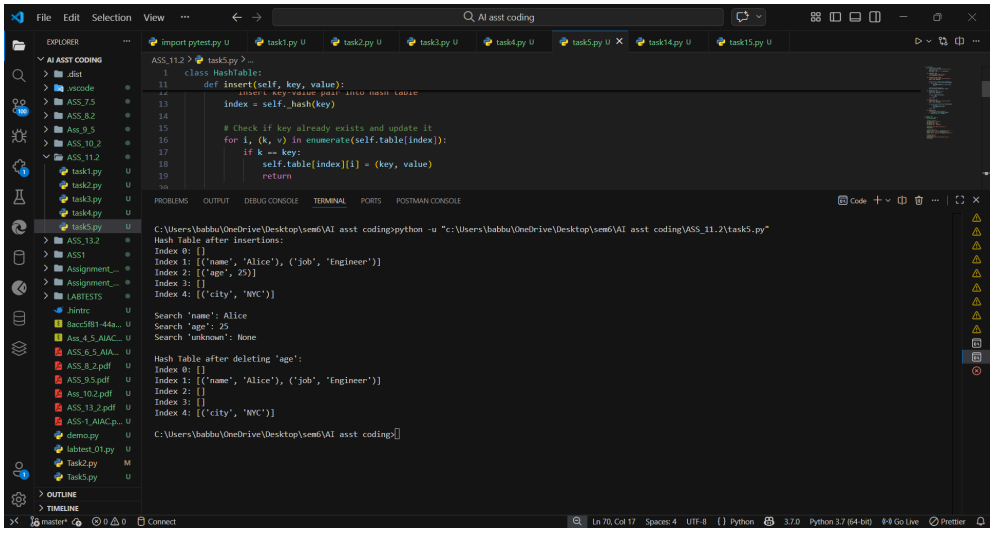
The result of in-order traversal in a BST always produces **sorted output**, which makes BST useful for searching and sorting operations.

> **Task Description -5 – (Hash Table Implementation)**
> **• Task: Create a hash table using AI with collision handling**
> **Expected Output:**
> **• Hash table supporting insert, search, and delete using chaining or open**

```
class HashTable:
    def __init__(self, size=10):
        """Initialize hash table with given size"""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate hash value for key"""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert key-value pair into hash table"""
        index = self._hash(key)

        # Check if key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return

        # Insert new key-value pair
        self.table[index].append((key, value))

    def search(self, key):
        """Search for value by key"""
        index = self._hash(key)

        for k, v in self.table[index]:
            if k == key:
                return v
```

```
        return None

    def delete(self, key):
        """Delete key-value pair from hash table"""
        index = self._hash(key)

        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index].pop(i)
                return True
        return False

    def display(self):
        """Display hash table contents"""
        for i, bucket in enumerate(self.table):
            print(f"Index {i}: {bucket}")


# Example usage
if __name__ == "__main__":
    ht = HashTable(5)

    # Insert operations
    ht.insert("name", "Alice")
    ht.insert("age", 25)
    ht.insert("city", "NYC")
    ht.insert("job", "Engineer")

    print("Hash Table after insertions:")
    ht.display()

    # Search operations
    print(f"\nSearch 'name': {ht.search('name')}")
    print(f"Search 'age': {ht.search('age')}")
    print(f"Search 'unknown': {ht.search('unknown')}")

    # Delete operation
    ht.delete("age")
    print("\nHash Table after deleting 'age':")
    ht.display()
```





EXPLAINATION

💡 A **Hash Table** is a data structure used to store data in the form of **key-value pairs**. It uses a **hash function** to convert a key into an index of an array where the value will be stored.
Hash tables provide **very fast access to data**, typically in **O(1) time complexity**.
However, sometimes two keys may generate the same index. This situation is called a **collision**.
To handle collisions, techniques such as **chaining** or **open addressing** are used. In this task, chaining is used where each index of the table contains a list of elements.
The hash table program supports operations such as:

- **Insert** – Add a key-value pair into the table.
- **Search** – Find a value using its key.
- **Delete** – Remove an element from the table.

Hash tables are widely used in applications such as **databases, caching systems, password storage, and dictionaries**.